

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Домашня контрольна робота
з дисципліни «Цифрова схемотехніка»
Тема: «Розроблення модулів арифметико-логічних блоків»

Виконали:
студенти групи ІО-46
Шеремета А. Р. , Попович М. О.
Номера залікових книжок: 4619, 4613

Перевірив:
Клименко І. А.

Тема: «Розроблення модулів арифметико-логічних блоків».

Мета: Розроблення та дослідження роботи арифметико-логічних блоків з загальними мікроопераціями. Дослідження формування ознак в арифметичних блоках загальними мікроопераціями. Нормалізація результату виконання арифметичних операцій. Навички розроблення рівня тестового коду.

Хід роботи

Вибраний номер залікової книжки – $4613_{10} = 1001000000101_2$

$h_7 = 0, h_6 = 0, h_5 = 0, h_4 = 0, h_3 = 1, h_2 = 0, h_1 = 1.$

h_2	h_3	h_1	Мікроінструкція АЛБ_МІ I_4I_3			
			00	01	10	11
0	1	1	$R - S - I + CI$	$R \& S$	$R + S + CI$	$R \vee S$

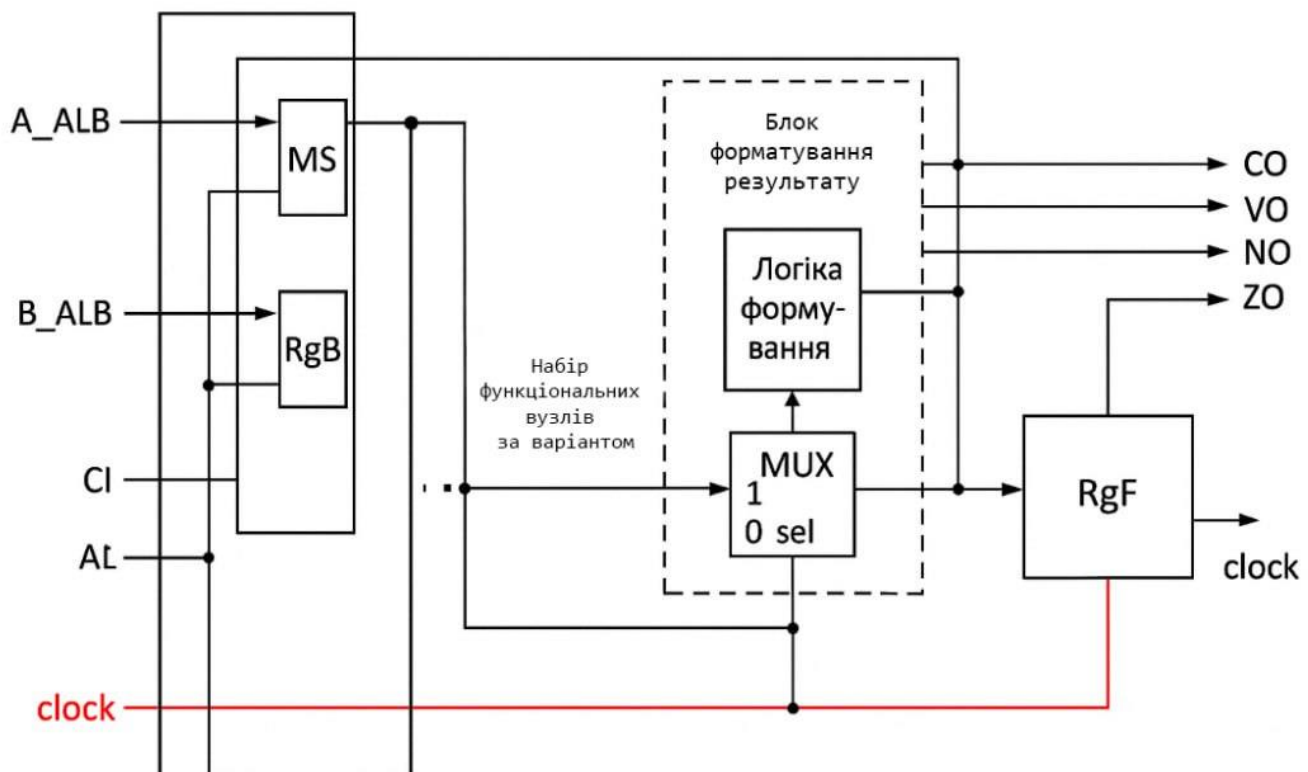


Рис. 1 – Функціональна модель ALB

Створення проєкту та тестування

Структура проєкту представлена на рис. 2.

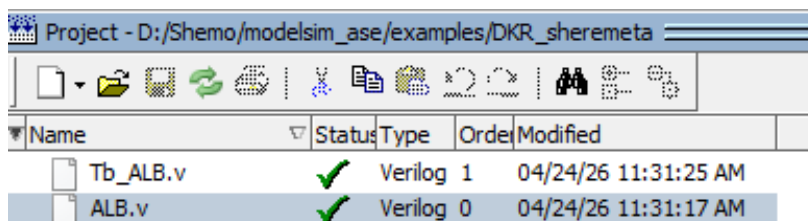


Рис. 2 – Файли проєкту

Роздруківка коду модуля АЛБ згідно з обраним варіантом (*ALB.v*):

```
module ALB (  
    input [3:0] R,  
    input [3:0] S,  
    input CI,  
    input [2:0] H,  
    input [1:0] I,  
    output reg [3:0] Y,  
    output reg [3:0] Y_norm,  
    output reg CO,  
    output reg VO,  
    output reg NO,  
    output reg ZO  
);  
  
// Попередній розрахунок усіх можливих результатів (паралельне виконання)  
wire [4:0] add_res = R + S + CI;  
wire [4:0] sub_res = R - S - 1 + CI;  
wire [3:0] and_res = R & S;  
wire [3:0] or_res = R | S;  
  
always @(*) begin  
    // Перевірка коду варіанта (h2=0, h3=1, h1=1)  
    if (H == 3'b011) begin  
        case (I)  
            2'b00: begin // Віднімання: R - S - 1 + CI  
                Y = sub_res[3:0];  
                CO = sub_res[4];  
                VO = (R[3] != S[3]) && (Y[3] != R[3]);  
            end  
            2'b01: begin  
                Y = and_res;
```

```

        CO = 0;
        VO = 0;
    end
    2'b10: begin
        Y = add_res[3:0];
        CO = add_res[4];
        VO = (R[3] == S[3]) && (Y[3] != R[3]);
    end
    2'b11: begin
        Y = or_res;
        CO = 0;
        VO = 0;
    end
    default: begin
        Y = 4'b0000;
        CO = 0;
        VO = 0;
    end
endcase
end else begin

    Y = 4'b0000;
    CO = 0;
    VO = 0;
end

NO = Y[3];
ZO = (Y == 4'b0000);

if (Y[3] != Y[2])
    Y_norm = {1'b0, Y[3:1]};
else
    Y_norm = Y;
end
endmodule

```

Роздруківка коду тестбенчу (*Tb_ALB.v*):

```
`timescale 1ns / 1ps
```

```
module tb_ALB;
```

```
    // Вхідні сигнали
```

```
    reg [3:0] R, S;
```

```
    reg CI;
```

```
    reg [2:0] H;
```

```
    reg [1:0] I;
```

```
    // Вихідні сигнали
```

```
    wire [3:0] Y, Y_norm;
```

```
    wire CO, VO, NO, ZO;
```

```
    // Створення екземпляра модуля
```

```
    ALB uut (
```

```
        .R(R), .S(S), .CI(CI), .H(H), .I(I),
```

```
        .Y(Y), .Y_norm(Y_norm),
```

```
        .CO(CO), .VO(VO), .NO(NO), .ZO(ZO)
```

```
    );
```

```
    initial begin
```

```
        $display("=== Тестування АЛБ (Варіант 4613, H=011) ===");
```

```
        // Встановлюємо правильний код варіанта
```

```
        H = 3'b011;
```

```
        CI = 1;
```

```
        // Тест 1: SUB (I=00) -> 8 - 4 - 1 + 1 = 4
```

```
        R = 4'b1000; S = 4'b0100; I = 2'b00; #10;
```

```
        $display("[00] SUB | R=%b S=%b | Y=%b Y_norm=%b | VO=%b", R, S, Y, Y_norm,  
VO);
```

```
        // Тест 2: AND (I=01) -> 1101 & 0111 = 0101
```

```
        R = 4'b1101; S = 4'b0111; I = 2'b01; #10;
```

```
        $display("[01] AND | R=%b S=%b | Y=%b Y_norm=%b", R, S, Y, Y_norm);
```

```
        // Тест 3: ADD (I=10) -> 5 + 3 + 1 = 9 (1001) - тут має бути нормалізація
```

```
        R = 4'b0101; S = 4'b0011; I = 2'b10; #10;
```

```
        $display("[10] ADD | R=%b S=%b | Y=%b Y_norm=%b | VO=%b", R, S, Y, Y_norm,  
VO);
```

```
        // Тест 4: OR (I=11) -> 0101 | 1000 = 1101
```

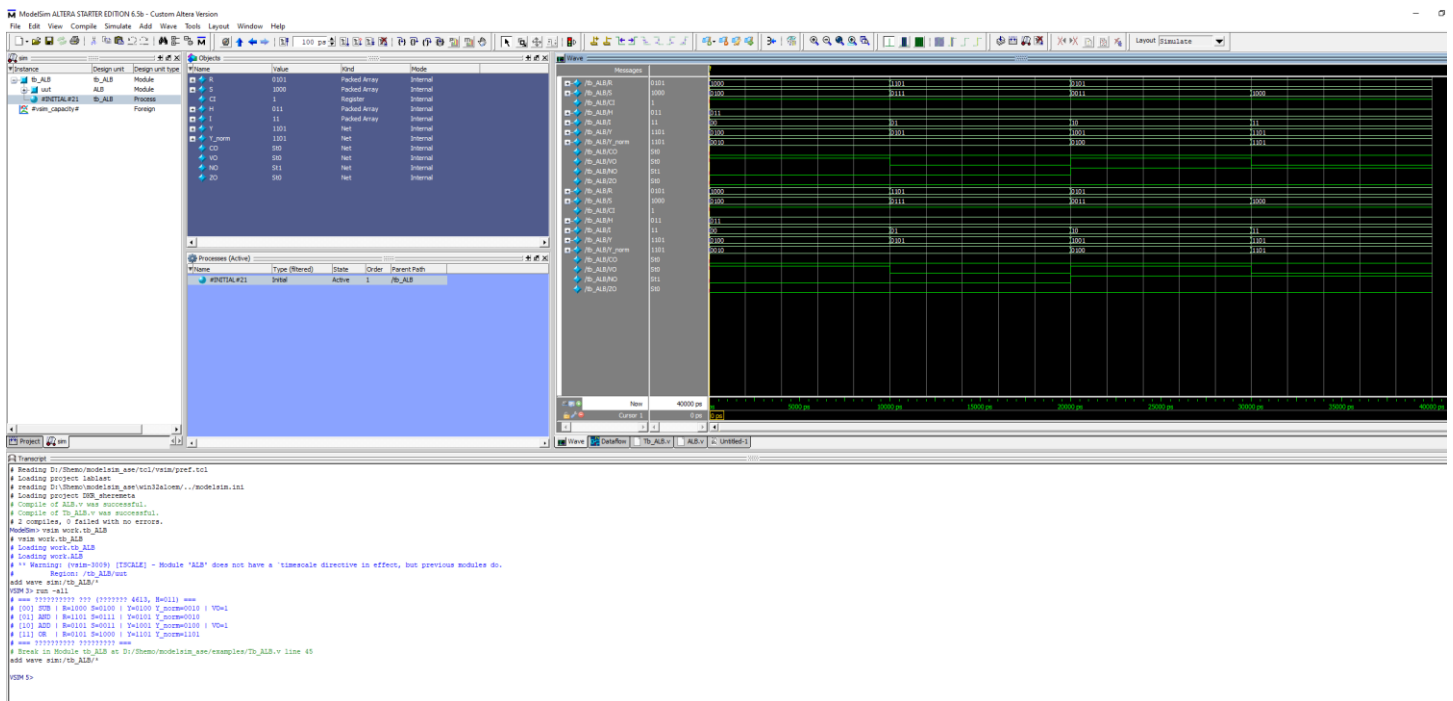



Рис. 5 – Загальний вигляд результатів тестування в середовищі ModelSim

Пояснення до виконання контрольної роботи

Арифметико-логічний блок (АЛБ) – це функціональний вузол, який виконує базові арифметичні та логічні операції над вхідними операндами. У даній реалізації АЛБ виконує одночасно чотири мікрооперації, а результат вибирається згідно з мікроінструкцією, закодованою у вигляді керуючих сигналів.

Керування виконанням операцій здійснюється сигналами h_2 , h_3 , h_1 (записані як один 3-бітний сигнал H) та 2-бітним кодом I (I_4I_3), який визначає вибір мікрооперації. Якщо $H = 011$, то можуть бути вибрані мікрооперації з таблиці нижче.

Таблиця відповідності I_4I_3 до мікрооперацій

Код I_4I_3	Операція	Опис
00	$R - S - 1 + CI$	Віднімання з використанням доповнення до двійки
01	$R \& S$	Побітове І (AND)
10	$R + S + CI$	Арифметичне додавання з переносом
11	$R \vee S$	Побітове АБО (OR)

Після виконання мікрооперації виконується додаткове завдання – нормалізація результату Y шляхом зсуву на один розряд праворуч, якщо старші два біти Y відрізняються (тобто $Y[3] \neq Y[2]$). Цей зсув зберігається у виході Y_{norm} .

Після обчислення основного результату встановлюються наступні прапори станів:

1. CO (*Carry Out*): встановлюється, якщо відбулося перенесення з найстаршого розряду.
2. VO (*Overflow*): встановлюється при арифметичному переповненні під час операцій додавання та віднімання.
3. NO (*Negative*): встановлюється, якщо результат має 1 у старшому біті (негативне число).
4. ZO (*Zero*): встановлюється, якщо результат дорівнює нулю.

Даний АЛБ забезпечує гнучке виконання різних операцій залежно від мікроінструкцій, і може бути використаний у цифрових процесорах, мікроконтролерах або емуляторах для обробки логіки та арифметики.

Результати експериментального дослідження ALB

У модулі ALB реалізовано механізм нормалізації результату Y шляхом зсуву праворуч, якщо $Y[3] \neq Y[2]$. Це дозволяє уникнути нестабільності при виході результату за межу знаковості та відновлює правильний формат числа при переповненні. Результат після нормалізації зберігається у змінній Y_{norm} . У таблиці операцій наведені приклади симуляції тестових значень.

Таблиця операцій тестових значень

Операція	Вхідні (R, S) при CI=1	Y	Y[3:2]	Зсув	Y_norm	Прапор VO	Опис
SUB [00]	1000, 0100	0100	01	Так	0010	1	$Y[3] \neq Y[2]$, нормалізація виконана (зсув)
AND [01]	1101, 0111	0101	01	Так	0010	0	$Y[3] \neq Y[2]$, нормалізація виконана (зсув)
ADD [10]	0101, 0011	1001	10	Так	0100	1	Переповнення $Y[3] \neq Y[2]$, нормалізація відновила результат
OR [11]	0101, 1000	1101	11	Ні	1101	0	$Y[3] = Y[2]$, зсув не потрібен

Під час симуляції роботи ALB з використанням ModelSim було проаналізовано часову поведінку модуля. Для кожної мікрооперації було використано затримку #10 ns. Загальний час симуляції склав близько 40 наносекунд.

Сигнали виходу (Y, Y_norm, CO, VO, NO, ZO) змінювались миттєво при зміні входів, що відповідає очікуваній поведінці комбінаційної логіки. Таким чином, розроблений ALB демонструє високу швидкодію.

Проведені експерименти підтвердили коректність роботи ALB та реалізованого механізму нормалізації. Зсув результату виконується чітко за умови незбіжності знакових розрядів, а результати роботи ALB є стабільними та прогнозованими.

Висновок

У ході виконання роботи було реалізовано арифметико-логічний блок (АЛБ), здатний виконувати чотири мікрооперації: віднімання з доповненням, побітове AND, арифметичне додавання з переносом та побітове OR. Вибір конкретної операції здійснюється мікроінструкцією, заданою сигналами H (h_2, h_3, h_1) та I ($I_4 I_3$). Успішно реалізовано додаткову обробку результату – нормалізацію шляхом зсуву на один розряд праворуч у разі, коли старші біти результату не збігаються. Робота АЛБ була ретельно перевірена за допомогою створеного тестбенчу на рівні RTL-симуляції у середовищі ModelSim. Усі результати розрахунків та відповідні прапори (CO, VO, NO, ZO) формуються коректно, що наочно підтверджується часовими діаграмами.